📌 Summary: Emoji Chat (Flyweight Pattern)

1. Goal of the Project

To save memory in a chat application. Instead of creating a new Emoji object for every single message, the application creates only one unique object for each distinct emoji type (e.g., 😊 and 💙) and reuses it.

2. Core Components

ChatAsset (Interface): The blueprint that forces any asset to have a display() method.

Emoji (Flyweight Object): Stores the Intrinsic State (the constant symbol, e.g., 😊).

EmojiFactory (The Cache): A HashMap that manages and reuses the emoji objects, preventing duplicates.

EmojiChatApp (Client): Simulates the chat and passes the Extrinsic State (dynamic data: username, messageText) to the display method at runtime.

1. Key Concept for the Exam/Discussion

Intrinsic State (Inside): Data that is constant and shared (symbol).

Extrinsic State (Outside): Data that changes and is passed from outside (username, messageText).

1. Code & Output at a Glance

Messages Sent: 4 messages.

Objects in Memory: Only 2 objects.

Efficiency: 50\% memory saving in this small test, and it scales up to millions of objects in real apps.